Assignment Report: Handwritten Digit Recognition using Support Vector Machines (SVM)

1. Introduction

The objective of this assignment is to develop an end-to-end data mining and machine learning pipeline capable of recognizing handwritten digits. The project consists of two primary components: a model training script that extracts patterns from a standard dataset, and an interactive Graphical User Interface (GUI) that allows a user to draw a digit and receive real-time predictions.

2. Methodology: Data Processing and Model Training (train_model.py)

The foundational relies on training a classification model to recognize spatial patterns in pixel data.

- **Dataset Acquisition:** The model utilizes the standard **MNIST dataset**, which contains grayscale images of handwritten digits (0-9).
- **Data Preprocessing (Dimensionality & Scaling):** Raw images in the dataset are 28 x 28 pixel matrices. For the Support Vector Machine (SVM) to process them, each image is "flattened" into a 1-dimensional array of 784 features. In a data mining context, each pixel represents an independent attribute.
    - **Normalization:** The pixel intensity values, originally ranging from 0 to 255, are divided by 255. Scaling the features to a [0, 1] range is a crucial preprocessing step for distance-based algorithms like SVMs to prevent features with larger numeric ranges from dominating the objective function.
- **Model Selection and Training:** The project utilizes a **Support Vector Machine (SVM)** classifier.
    - A **Radial Basis Function (RBF) Kernel** is applied (kernel='rbf'). This allows the SVM to map the input features into a higher-dimensional space to find non-linear decision boundaries, which is essential for complex visual data like handwriting.
    - To optimize training time and computational resources, the model is fit on a representative subset of the data (the first 10,000 samples).
- **Model Persistence:** Once trained, the model's mathematical state is serialized and saved to disk (mnist_svm.pkl) using the joblib library. This allows the application to perform predictions later without needing to retrain the model.

3. Implementation: The Interactive GUI (digit_gui.py)

To make the data mining model accessible, a custom graphical interface was developed using the Tkinter library. The primary challenge of the GUI is ensuring that user-generated data perfectly mimics the format of the training data.

- **Data Capture:** The user draws on a 280 x 280 pixel canvas. The interface is specifically designed with a black background and white drawing brush to match the negative-space format of the MNIST dataset.
- **Real-time Transformation Pipeline:** When the "Predict" button is clicked, the application applies a strict preprocessing pipeline to the user's drawing:
    1. **Downsampling:** The high-resolution 280 x 280 canvas image is resized down to the 28 x 28 dimension expected by the model.

2. **Array Conversion & Scaling:** The image is converted to a NumPy array and normalized (divided by 255) to match the [0, 1] scale of the training data.
3. **Reshaping:** The 2D array is flattened into a 1 x 784 vector.

- **Inference:** This preprocessed 784-feature vector is passed to the pre-loaded SVM model. The model calculates the decision function and outputs the integer class (0-9) that represents the most mathematically probable digit, which is then displayed to the user.

4. Test Cases

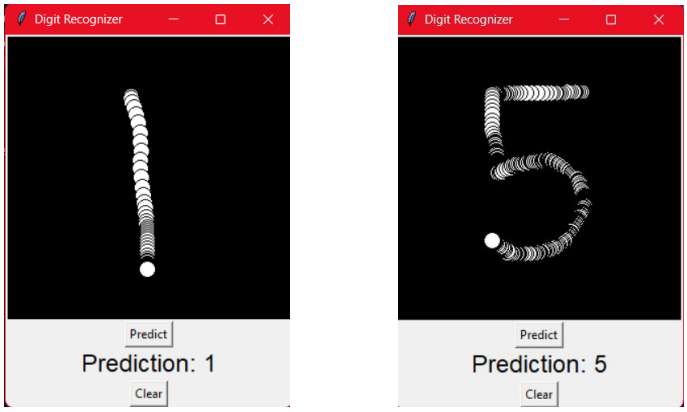

Figure 1: Successful Classification Case

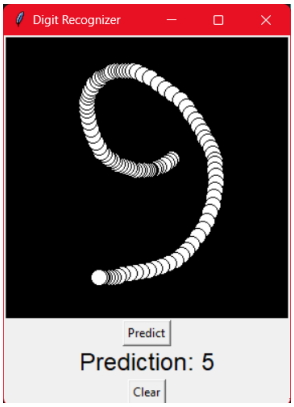

Figure 2: Misclassification Analysis

The misclassification in Figure 2 may be attributed to the Support Vector Machine's sensitivity to the thickness of the lines or the specific 'RBF' kernel parameters, which might struggle with non-standard handwriting styles not heavily represented in the training subset.

-Avirup Roy
[232010007010]
CSE, 6th sem